

Transaction commit and Rollback Lab Exercise

Common setup: Bank schema

All students to first create and populate this schema.

sql

```
CREATE TABLE Accounts (  
    acc_id    INT PRIMARY KEY,  
    acc_name  VARCHAR(50),  
    balance   INT  
);
```

INSERT INTO Accounts **VALUES**

```
(1, 'Alice', 10000),  
(2, 'Bob', 8000),  
(3, 'Carol', 5000);
```

Exercise 1 – Basic transaction with COMMIT

Task

1. Start a transaction.
2. Transfer 2000 from Alice (1) to Bob (2).
3. Check balances inside the transaction.
4. Commit the transaction.
5. Verify final balances after commit.

Expected student SQL

sql

-- Step 1: start transaction

BEGIN; *-- or START TRANSACTION;*

-- Step 2: transfer 2000 from Alice to Bob

UPDATE Accounts

SET balance = balance - 2000

WHERE acc_id = 1;

UPDATE Accounts

SET balance = balance + 2000

WHERE acc_id = 2;

-- Step 3: check inside the same transaction

SELECT * FROM Accounts;

-- Step 4: commit

COMMIT;

-- Step 5: verify after commit

SELECT * FROM Accounts;

Key points to explain

- The transfer must be **atomic**: either both updates succeed, or none.
- After commit, new balances are permanently stored.

Exercise 2 – Transaction with ROLLBACK

Task

1. Start a transaction.
2. Mistakenly transfer 5000 from Bob (2) to Carol (3).
3. Realize the mistake and roll back.
4. Show that balances are unchanged compared to before.

Expected student SQL

sql

-- Assume current balances after Exercise 1

SELECT * FROM Accounts;

BEGIN;

UPDATE Accounts

SET balance = balance - 5000

WHERE acc_id = 2;

UPDATE Accounts

SET balance = balance + 5000

WHERE acc_id = 3;

-- Check inside transaction

SELECT * FROM Accounts;

-- Decide to cancel

ROLLBACK;

-- Check after rollback

SELECT * FROM Accounts;

Key points

- ROLLBACK undoes **all** changes made in the current transaction.
- Good for demonstrating error handling / wrong updates.